

Managing Interesting Rules in Sequence Mining

Myra Spiliopoulou

Institut für Wirtschaftsinformatik, Humboldt-Universität zu Berlin
Spandauer Str. 1, D-10178 Berlin
myra@wiwi.hu-berlin.de
<http://www.wiwi.hu-berlin.de/~myra>

Abstract. The goal of sequence mining is the discovery of interesting sequences of events. Conventional sequence miners discover only frequent sequences, though. This limits the applicability scope of sequence mining for domains like error detection and web usage analysis. We propose a framework for discovering and maintaining interesting rules and beliefs in the context of sequence mining. We transform frequent sequences discovered by a conventional miner into sequence rules, remove redundant rules and organize the remaining ones into interestingness categories, from which unexpected rules and new beliefs are derived.

1 Introduction

Data miners pursue the discovery of new knowledge. But knowledge based solely on statistical dominance is rarely new. The expert needs means for either instructing the miner to discover only interesting rules or for ranking the mining results by “interestingness” [7].

Tuzhilin et al propose interestingness measures based on the notion of belief [1, 6]. A belief reflects the expert’s domain knowledge. Mining results that contradict a belief are more interesting than those simply confirming it. Hence, they propose methods to guide the miner in the discovery of interesting rules only.

Most conventional miners did not foresee the need for such a guidance, when they were developed, so that much research has focussed on the filtering of the mining results in a postmining phase [4]. Sequence miners are no exception: [2, 5, 10], the goal is the discovery of frequent sequences, which are built by expanding frequent subsequences in an incremental way. The sequence miner WUM, presented in [9, 8], does support more flexible measures than frequency of appearance. Still missing is a full system that administers the beliefs, compares and categorizes the mining results towards them and uses the finally select rules and beliefs as an input to the next mining session.

In this study, we propose such a postmining environment. Its goal is to prune and categorize the patterns discovered by the miner, confirm or update the beliefs of the expert and classify unexpected patterns into several, distinct categories. Then, the expert may probe potentially unexpected rules using her updated beliefs or *expected rules* as a basis. This framework adds value to the functionality of existing miners, helps the expert to gain an overview of the results, to update her system of beliefs and to design the next mining session effectively.

2 Sequence Rules

In sequence mining, the events constituting a *frequent sequence* need not be adjacent [2]. Thus, a frequent sequence does not necessarily ever occur in the log. We remove this formal oddity by introducing the notion of generalized sequence or “g-sequence” as a list of events *and wildcards* [8].

Definition 1. A “g-sequence” g is an ordered list $g_1 * g_2 * \dots * g_n$, where $g_1, \dots, g_n$ are elements from a set $\in U$ and $*$ is a wildcard. The number of non-wildcard elements in g is the length of g , $\text{length}(g)$.

A sequence in the transaction log $\mathcal{L}$ matches a g-sequence if it contains the non-wildcard elements of the g-sequence in that order. Then:

Definition 2. Let $\mathcal{L}$ be a transaction log and let $g \in U_+^*$ be a g-sequence. The hits of g , $\text{hits}(g)$, is the number of sequences in $\mathcal{L}$ that match g .

2.1 Rules for g-Sequences

A conventional miner discovers frequent sequences, or “g-sequences” in our terminology. A “sequence rule” is built by splitting such a g-sequence into two adjacent parts: the LHS or premise and the RHS or conclusion. We denote a sequence rule as $\langle LHS, RHS \rangle$ or $LHS \hookrightarrow RHS$.

Definition 3. Let $\zeta = \langle g, s \rangle$ be a sequence rule, with $n = \text{length}(g)$ and $m = \text{length}(s)$. Further, let $|\mathcal{L}|$ denote the cardinality of the transaction log $\mathcal{L}$.

- “support” of ζ : $\text{support}(\zeta) \equiv \text{support}(g \cdot s) = \frac{\text{hits}(g \cdot s)}{|\mathcal{L}|}$
- “confidence” of ζ : $\text{confidence}(\zeta) \equiv \text{confidence}(g, s) = \frac{\text{support}(g \cdot s)}{\text{support}(g)}$
- “improvement” of ζ : $\text{improvement}(\zeta) \equiv \text{improvement}(g, s) = \frac{\text{confidence}(g, s)}{\text{support}(s)}$

where $g \cdot s$ denotes the concatenation of g and s .

We use the notion of support as in [2]. We borrow the concepts of confidence and improvement from the domain of association rules’ discovery [3]. An improvement value less than 1 means that the RHS occurs independently of the LHS and the rule is not really interesting.

2.2 Extracting and Pruning Sequence Rules

We now turn to the problem of producing rules as the results of the mining process. A frequent sequence generates several rules by shifting events from the LHS to the RHS. Many of those rules are of poor statistics and must be removed.

Let $g = g_1 * g_2 * \dots * g_n$ be a frequent sequence output by a conventional sequence miner. To compute its statistics, we observe that since g is frequent, then all its elements $g_1, g_2, \dots, g_n$ and all its order preserving subsequences, like $g_1 * g_2, g_2 * g_3, g_1 * g_2 * g_3$ are also frequent. Then, the support values are known and the confidence and improvement values for any sequence rules containing those subsequences can be computed.

Input: The set $\mathcal{G}$ of all g-sequences discovered by the miner,
a confidence threshold t_c and
an improvement threshold t_{impr} (default: 1)
Output: For each $g \in \mathcal{G}$, all “acceptable” partitionings of g into an LHS and an RHS
Algorithm BuildSRules: For each $g \in \mathcal{G}$ do:
For $i = n - 1, \dots, 1$ do:
Set $LHS = g_1 * \dots * g_i$ and $RHS = g_{i+1} * \dots * g_n$
If the $confidence(LHS, RHS) < t_c$ then discard the rule
Else-If $improvement(LHS, RHS) < t_{impr}$ then discard the rule

Fig. 1. Selectively building sequence rules

Building Sequence Rules from a g-Sequence. From a g-sequence $g = g_1 * \dots * g_n$ and for any i between 2 and $n - 1$, we can build two sequence rules $\zeta_1 : g_1 * \dots * g_{i-1} * \hookrightarrow g_i * \dots * g_n$ and $\zeta_2 : g_1 * \dots * g_i * \hookrightarrow g_{i+1} * \dots * g_n$. From Def. 3, we can see that the confidences of ζ_1 and ζ_2 are ratios with the same nominator $support(g)$ and with denominators $support(g_1 * \dots * g_{i-1})$ and $support(g_1 * \dots * g_i)$ respectively. Since all sequences containing $g_1 * \dots * g_i$ also contain $g_1 * \dots * g_{i-1}$, $confidence(\zeta_1) \leq confidence(\zeta_2)$.

Thus, when shifting elements from the LHS to the RHS, the support of the LHS increases and the rule’s confidence decreases. For the same reason, the support of the RHS decreases, so that the improvement changes non-monotonically.

High values of improvement are desirable. However, this measure endeavours rules with rare RHS. So it must be used in combination with the confidence measure. Thus, our **buildSRules** algorithm in Fig. 1 only builds rules with confidence higher than a threshold t_c and order them by improvement. Rules with improvement less than 1 or some higher threshold t_{impr} are eliminated. Default thresholds can be provided for both t_c, t_{impr} . However, analysts can be expected to provide such values, as is usual for association rules’ discovery.

Pruning Sequence Rules by Comparison. After building a first set of sequence rules with **buildSRules**, we remove rules that are implied by statistically stronger ones. We consider rules that have the same RHS and overlapping LHS contents. The algorithm **pruneSRules** is shown in Fig. 2.

3 Beliefs and Unexpectedness

Thus far, we have established a set of sequence rules and removed those members that had poor statistics. We now build a system of beliefs and categorize the sequence rules according to their relationships to the beliefs.

3.1 Beliefs over Sequences

Informally, a belief is a sequence rule assumed to hold on the data. This assumption is expressed in the form of value intervals that restrict the support, confidence or improvement of the elements appearing in the rule.

Input: The set $\mathcal{SR}$ of all sequence rules built by buildSRules

Output: A subset $\mathcal{SR}_{out}$ of $\mathcal{SR}$

Algorithm PruneSRules:

1. Group sequence rules by RHS
2. Sort the sequence rules of each group by descending LHS length
3. For each rule $\zeta = lhs \hookrightarrow rhs$:
 - Find all sequence rules of the form $lhs_2 \hookrightarrow rhs$ such that:
 - $lhs \supseteq lhs_1 \cdot lhs_2$
 - There is a sequence rule $lhs_1 \hookrightarrow lhs_2$
 - For each triad $(\zeta, lhs_1 \hookrightarrow lhs_2, lhs_2 \hookrightarrow rhs)$:
 - Let** $c_1 = confidence(\zeta)$,
 $c_2 = confidence(lhs_1 \hookrightarrow lhs_2)$, $c_3 = confidence(lhs_2 \hookrightarrow rhs)$
 - If** $c_1 < \min\{c_2, c_3\}$ then remove ζ
 (ζ is implied by the other rules and has lower confidence than them)
 - Else-If** $c_3 < \min\{c_1, c_2\}$ then remove $lhs_2 \hookrightarrow rhs$
 (rhs is predicted by ζ , while lhs_2 is predicted by $lhs_1 \hookrightarrow lhs_2$, both with a higher confidence)
 - Else-If** $c_3 > \max\{c_1, c_2\}$, then retain $lhs_2 \hookrightarrow rhs$ and the one of the two other rules with the largest improvement

Fig.2. Algorithm for sequence rule pruning

Definition 4. A “belief” β over g -sequences is a tuple $\langle lhs, rhs, CL, C \rangle$:

- $lhs \hookrightarrow rhs$ is a sequence rule.
- CL is a conjunction of “constraints” on the statistics of lhs .
- C is a conjunction of constraints involving elements of lhs and rhs .
- At least one of CL and C is not empty.

A “constraint” is an expression of the form $stats(x, y) \circ c$, where x, y are g -sequences contained in lhs or rhs , $stats()$ is one of $support()$, $confidence()$ and $improvement()$, while the symbol $\circ$ denotes a comparison to a constant c in the value range of $stats()$.

For example, let $\beta = \langle a * b, c, CL, C \rangle$ be a belief with $CL = (support(a * b) \geq 0.4 \wedge confidence(a, b) \geq 0.8)$ and $C = (confidence(a * b, c) \geq 0.9)$. This belief states that the LHS of the sequence rule $a * b \hookrightarrow c$ should appear at least in 40% of the log sequences, the confidence of b given a should be at least 0.8, while the RHS confidence should be at least 0.9.

In most research in the area of beliefs and interestingness, it is assumed that the beliefs are known in advance. We rather expect that most beliefs will be identified during a postmining phase, as described in section 4.

3.2 Expected and Unexpected Sequence Rules

Having defined the notion of belief over sequences, we now categorize sequence rules on the basis of their collision against beliefs.

Definition 5. Let $\zeta = lhs \hookrightarrow rhs$ be a sequence rule found by mining over the transaction log $\mathcal{L}$, such that: $support(lhs) = s$, $support(\zeta) = s'$, $confidence(\zeta) = c$ and $improvement(\zeta) = i$ according to Def. 3.

Let $\beta = \langle lhs', rhs, CL, C \rangle \in \mathcal{B}$ be a belief such that $lhs = lhs' \cdot y$ (where y may be empty). The match of ζ towards β , $match(\beta, \zeta)$, is the pair of predicates $(CL(\zeta), C(\zeta))$, where:

- $CL(\zeta)$ is the conjunction $CL \wedge (support(lhs) = s)$
- $C(\zeta)$ is the conjunction $C \wedge (support(lhs) = s) \wedge support(\zeta) = s' \wedge confidence(\zeta) = c \wedge improvement(\zeta) = i$.

Here we compare the sequence rule to a belief. This is only possible for beliefs and rules having similar contents. Otherwise, the notion of match is undefined, i.e. “there is no match”.

Definition 6. Let $\mathcal{B}$ be the collection of beliefs defined over $\mathcal{L}$ thus far. Further, let $\zeta = lhs \hookrightarrow rhs$ be a sequence rule found by mining over $\mathcal{L}$.

Then ζ is “expected” if there is a $\beta = \langle lhs', rhs, CL, C \rangle \in \mathcal{B}$, such that $match(\beta, \zeta)$ is defined and $CL(\zeta) \wedge C(\zeta) = true$. If no such belief exists, ζ is “unexpected”.

Thus, a sequence rule is *expected* if it conforms to a belief in terms of statistics and content. Depending on the reason that makes a rule unexpected, we categorize *unexpected* rules as follows:

Definition 7. Let $\mathcal{B}$ be the collection of beliefs over a log $\mathcal{L}$ and let $\zeta = \langle lhs, rhs \rangle$ be an unexpected sequence rule.

- ζ is “statistically deviating” if there is a belief with same LHS and RHS $\beta = \langle lhs, rhs, CL, C \rangle \in \mathcal{B}$, such that:
 $CL(\zeta) \wedge C(\zeta) = false$
- ζ “makes unexpected predictions” if there is a $\beta = \langle lhs, rhs', CL, C \rangle \in \mathcal{B}$, such that:
 1. $rhs' \neq rhs$ but $length(rhs') = length(rhs)$
We build a temporary belief $\beta_r = \langle lhs, rhs, CL, C_r \rangle$ by 1-to-1 replacing references to rhs' in C with references to rhs .
 2. In the $match(\beta_r, \zeta)$ it holds that $CL(\zeta) \wedge C_r(\zeta) = true$.
- ζ “makes unexpected assumptions” if there is a belief with the same RHS $\beta = \langle lhs', rhs, CL, C \rangle \in \mathcal{B}$, such that:
 1. $lhs' \neq lhs$ and $length(lhs) - length(lhs') = n > 0$
 - (a) There is a temporary belief $\beta_l = \langle lhs, rhs, CL_l, C_l \rangle$ built replacing references to lhs' in CL and C with references to a subsequence of lhs of the same length.
 - (b) In the $match(\beta_l, \zeta)$ it holds that $CL_l(\zeta) \wedge C_l(\zeta) = true$

If none of the above cases holds, then ζ is called “new”.

We group unexpected rules by the semantics of their “unexpectedness”. To test whether a rule makes unexpected predictions or assumptions, we search for all beliefs having the same LHS, resp. RHS, and build temporary beliefs that match with the rule. It should be stressed that a rule is unexpected if there is no belief that makes it expected.

Input: A set of beliefs $\mathcal{B}$, the set of frequent sequences $\mathcal{S}$

Output: The modified set of beliefs $\mathcal{B}$ and 5 sets of rules:

1. the set of expected rules $\mathcal{E}$
2. the set of statistically deviating unexpected rules $\mathcal{D}$
3. the set of unexpected rules making unexpected predictions $\mathcal{P}$
4. the set of unexpected rules making unexpected assumptions $\mathcal{A}$
5. the set of unexpected new rules $\mathcal{N}$

Algorithm PostMine:

1. Invoke **BuildSRules** to construct the sequence rules from the frequent sequences
2. Invoke **PruneSRules** to reduce the set of sequence rules into $\mathcal{SR}_{out}$.
3. By comparing the beliefs in $\mathcal{B}$ to the sequence rules in $\mathcal{SR}_{out}$, identify the expected rules and place them to $\mathcal{E}$.
4. For each $\zeta \in \mathcal{SR}_{out} - \mathcal{E}$:
 - If** there is a belief $\beta \in \mathcal{B}$ violated by ζ according to Def. 7, then:
 - if** ζ is statistically deviating then
 - add ζ to $\mathcal{D}$
 - else-if** ζ makes unexpected predictions then
 - add ζ to $\mathcal{P}$
 - suggest extending β or replacing it by ζ
 - else-if** ζ makes unexpected assumptions then
 - add ζ to $\mathcal{A}$
 - suggest transforming ζ to a belief
 - Else** add ζ to $\mathcal{N}$

Fig. 3. Algorithm PostMine for rule categorization

4 Organizing Sequence Rules by Unexpectedness

We now present our complete algorithm for the categorization of sequence rules by unexpectedness. Its input consists of a set of beliefs and a set of frequent sequences discovered by a conventional miner. Initially, the set of beliefs may be empty or rudimentary: it is reconstructed by the end of the post-mining phase by adding new beliefs and removing outdated ones.

The algorithm **PostMine** is shown in Fig. 3. It first invokes **buildSRules** (Fig. 1) to generate sequence rules from frequent sequences and then activates **pruneSRules** (Fig. 2) for rule filtering. The remaining sequence rules are categorized by (un)expectedness. For sequence rules making unexpected predictions or assumptions, **PostMine** suggests a change in the belief system.

The set E of expected rules output by **PostMine** can now be used as input to the mechanism for rule negation proposed in [6] or be generalized into pattern templates as proposed in [1] for the discovery of further interesting rules.

5 Conclusions

We have proposed a complete post-mining mechanism for the extraction of interesting sequence rules from frequent sequences according to a non-fixed set of

beliefs. Our **PostMine** model transforms frequent sequences into a set of sequence rules and filters this set using statistical measures and heuristics based on content overlap and transitivity. The remaining sequence rules are categorized on the basis of their relationship to a collection of beliefs. Some of the expected rules become themselves beliefs, probably replacing old ones.

PostMine helps in bringing order into the vast set of frequent sequences a miner can generate. This is fundamental for proper rule maintenance and for the stepwise formulation of a system of beliefs that combines the expert's background knowledge with the rules hidden in the data. The expert may revise her knowledge by studying the unexpected rules of each category and considering the suggestions of our algorithm.

We are currently implementing **PostMine** and intend to use it together with our sequence miner **WUM**. In this coupling, we want to investigate the automated extraction of beliefs based on variables, as supported by **WUM**, rather than constants. This would lead to a smaller set of generic beliefs, which the expert can inspect and manipulate easier. We are further interested in the reduction of a set of sequence rules into a minimal set with still reliable statistics.

References

1. Gediminas Adomavicius and Alexander Tuzhilin. Discovery of actionable patterns in databases: The action hierarchy approach. In *KDD*, pages 111–114, Newport Beach, CA, Aug. 1997.
2. Rakesh Agrawal and Ramakrishnan Srikant. Mining sequential patterns. In *ICDE*, Taipei, Taiwan, Mar. 1995.
3. Michael J.A. Berry and Gordon Linoff. *Data Mining Techniques: For Marketing, Sales and Customer Support*. John Wiley & Sons, Inc., 1997.
4. Alex Freitas. On objective measures of rule surprisingness. In *PKDD'98*, number 1510 in LNAI, pages 1–9, Nantes, France, Sep. 1998. Springer-Verlag.
5. Heikki Mannila and Hannu Toivonen. Discovering generalized episodes using minimal occurrences. In *KDD'96*, pages 146–151, 1996.
6. Balaji Padmanabhan and Alexander Tuzhilin. A belief-driven method for discovering unexpected patterns. In *KDD'98*, pages 94–100, New York City, NY, Aug. 1998.
7. Gregory Piatetski-Shapiro and Christopher J. Matheus. The interestingness of deviations. In *AAAI'94 Workshop Knowledge Discovery in Databases*, pages 25–36. AAAI Press, 1994.
8. Myra Spiliopoulou. The laborious way from data mining to web mining. *Int. Journal of Comp. Sys., Sci. & Eng., Special Issue on "Semantics of the Web"*, Mar. 1999.
9. Myra Spiliopoulou and Lukas C. Faulstich. **WUM**: A Tool for Web Utilization Analysis. In *extended version of Proc. EDBT Workshop WebDB'98*, LNCS 1590. Springer Verlag, 1999.
10. Mohammed J. Zaki. Fast mining of sequential patterns in very large databases. Technical Report 668, University of Rochester, 1997.